GraphAide: Advanced Graph-Assisted Query and Reasoning System

Sumit Purohit, George Chin, Patrick S Mackey, Joseph A Cottam
Pacific Northwest National Laboratory
Richland, Washington 99352
{sumit.purohit, george.chin, patrick.mackey, joseph.cottam}@pnnl.gov

2024 IEEE International Conference on Big Data (BigData) | 979-8-3503-6248-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/BigData62323.2024.10825705

***Abstract*—Curating knowledge from multiple siloed sources that contain both structured and unstructured data is a major challenge in many real-world applications. Pattern matching and querying represent fundamental tasks in modern data analytics that leverage this curated knowledge. The development of such applications necessitates overcoming several research challenges, including data extraction, named entity recognition, data modeling, and designing query interfaces. Moreover, the explainability of these functionalities is critical for their broader adoption.**

The emergence of Large Language Models (LLMs) has accelerated the development lifecycle of new capabilities. Nonetheless, there is an ongoing need for domain-specific tools tailored to user activities. The creation of such digital assistants has gained considerable traction in recent years, with LLMs offering a promising avenue to develop such assistants utilizing domain-specific knowledge and assumptions.

In this context, we introduce an advanced query and reasoning system, *GraphAide*, which constructs a knowledge graph (KG) from diverse sources and allows to query and reason over the resulting KG. *GraphAide* harnesses both the KG and LLMs to rapidly develop domain-specific digital assistants. It integrates design patterns from retrieval augmented generation (RAG) and the semantic web to create an agentic LLM application. *GraphAide* underscores the potential for streamlined and efficient development of specialized digital assistants, thereby enhancing their applicability across various domains.

I. Introduction

Large Language Models (LLMs) [1] represent the cutting edge of generative artificial intelligence (GenAI) and machine learning research and development. They have been adopted at an extraordinary rate across a range of disciplines, including scientific research, engineering, economics, and social sciences. By enabling domain experts to utilize pre-trained, ready-to-use models, LLMs have democratized the application development process, catalyzing innovations in both scientific and technological domains. The development of LLM-based applications remains a dynamic area of research, offering superior performance in tasks such as summarization, correlation, and inference across various input sources [2].

Knowledge Graphs (KGs) are formal representations of key concepts as entities and the relationships between them [3], [4]. KGs serve as powerful tools that can be employed in diverse user environments as foundational reference knowledge sources. However, constructing these KGs presents substantial challenges due to the scale of input data, the heterogeneity of use-case-specific concepts, and the costs associated with KG construction. LLMs have demonstrated higher accuracy in extracting relevant entities and forming relationships, significantly contributing to KG construction [5].

A major challenge in developing accurate and consistent capabilities based on LLMs is *hallucination* [6], which occurs when LLMs generate non-existent facts in response to user queries. The memorization of training data by LLMs and the subsequent reliance on corpus-based heuristics are significant factors contributing to hallucinations [7]. Addressing this issue is crucial for the reliability and trustworthiness of LLM-derived applications.

Retrieval-augmented generation (RAG) [8] is a well-established design pattern aimed at mitigating hallucination in Large Language Models (LLMs) by providing additional context during the generation of responses to user queries. This additional *context* is domain-specific and acts as *grounding* for the model's generative capabilities. A foundational RAG-based system employs a vector database to store *embeddings* of a domain-specific knowledge corpus and utilizes semantic similarity algorithms to retrieve the context relevant to the user query. The semantic similarity algorithms measure how closely the meanings of two concepts align. This context is then combined with the user query and forwarded to the LLM, serving as a guardrail for its generation process. The RAG-based approach has demonstrated efficacy in reducing hallucination by constraining the LLM's generative output to the localized region of the provided context.

However, a vector search-based method is constrained by the semantic similarity between the query and the corpus. It fails to leverage the structural context, such as the relationships between documents within the corpus, their metadata, or the associated reasoning. Knowledge Graphs (KGs) offer a robust solution to these limitations by efficiently storing domain-specific information and establishing relationships between information sources dispersed across documents in the corpus. KGs can also significantly enhance various components of an LLM-based application, including retrieval, summarization, and inference.

By integrating KGs into the design pattern, the system can utilize both the semantic and structural context, thereby improving the accuracy and consistency of the generated responses. This comprehensive approach enables a more nuanced understanding and utilization of the knowledge corpus, offering a sophisticated strategy to further reduce hallucination

979-8-3503-6248-0/24/$31.00©2024 IEEE
Authorized licensed use limited to: Kookmin University. Downloaded on August 31,2026 at 11:15:48 UTC from IEEE Xplore. Restrictions apply.

메모

and enhance the application's overall performance.

We introduce GraphAide, an advanced capability based on Large Language Models (LLMs) that offers insights into domain-specific data and allows users to ask natural language questions. GraphAide presents a methodology and reference architecture to develop advanced GenAI applications using different components such as LLMs, vector database, and graph database. GraphAide employs a modular, extensible, and user-friendly retrieval-augmented generation (RAG) approach, incorporating both vector and graph databases. The novelty of GraphAide lies in its comprehensive design for integrating GenAI and semantic web technologies. GraphAide curates multi-source data and constructs ontology-guided knowledge graphs to address the challenges faced by baseline LLM-only applications. The platform also provides efficient data persistence, enabling interactions via a natural language question-answering interface.

GraphAide combines a robust graph matching approach with the semantic similarity provided by vector databases to enhance the accuracy and explainability of LLM-generated responses. Developed as an extensible agentic RAG application, GraphAide reuses data ingestion, entity disambiguation, storage, query, and reasoning components throughout the application lifecycle. Key capabilities of GraphAide include:

- **Ontology-guided Knowledge Graph construction:** Handles multi-value, multi-source, temporal attributes from common data formats such as .txt, .html, .pdf, etc.
- **Generation and maintenance of embeddings:** Configures vector databases for storage and retrieval, maintaining multiple vector databases and dynamically selecting sources based on user queries.
- **Template-based ontology configuration:** Enables dynamic input ontology configuration and generation of multiple KG versions for a given source dataset.
- **Enhanced context generation:** Combines vector-based retrieval with graph-based subgraph matching to improve the relevancy of LLM-generated responses.
- **External function execution capability:** Enriches context by retrieving multi-modal data sources, such as temporal data from time-series databases.

GraphAide represents a significant advancement in the development of domain-specific digital assistants, integrating state-of-the-art GenAI and semantic technologies for improved performance and extensibility. Section II provides an overview of related work, while Section III details the GraphAide architecture along with its key components. Section IV showcases a real-world use case, presenting experimental results and evaluation metrics.

II. Related Work

The integration of generative artificial intelligence (GenAI) and semantic technologies is an active and burgeoning area of research. Existing Knowledge Graph (KG)-based LLM capabilities often focus on specific applications. A baseline RAG approach [9] transforms unstructured data into multiple chunks, creates embeddings for these chunks, and stores them in a vector database. These embeddings are then used to perform semantic similarity matching for an input user query, retrieving the most similar chunks. This retrieval process constructs the *context* to augment user queries sent to the LLM. The RAG approach has shown promising results in addressing challenges such as hallucination, outdated knowledge, and the explainability of generated responses. However, the baseline RAG is limited by its reliance on semantic similarity.

GraphRAG [10] advances this field by presenting a query-focused summarization task (QFS) that builds efficient text indices over a KG. GraphRAG leverages community detection to partition the graph index and uses it to generate "global answers" to user queries. Other approaches, such as the work by Trajanoska et al. [11], focus on creating KGs from sustainability-related text and evaluating their quality with specialized models for joint entity and relation extraction. Similarly, Ban et al. [12] introduce a set of prompts to extract causal graphs for causal structural learning. A common challenge with these approaches is their limited extensibility, as they do not leverage state-of-the-art open-source application development frameworks. There exists a significant opportunity to utilize KGs more effectively in various phases of LLM-based application development and usage.

III. Architecture

GraphAide is a retrieval-augmented generation (RAG)-based system that employs multiple LLM instances and knowledge graphs (KGs) to perform diverse tasks. An *Agent*-based LLM application leverages its reasoning capabilities to determine the appropriate sequence of actions in response to user queries. Unlike hardcoded instructions (i.e., *chains*), an *agentic application* can comprehend the LLM's response and construct subsequent queries dynamically. GraphAide is a hybrid application leveraging both chains and agents to handle a variety of user queries. GraphAide's hybrid approach enables the system to effectively manage a wide array of user queries, ensuring accurate and contextually relevant responses.

Figure 1 details GraphAide architecture and highlights use of ontology, vector database, and multiple LLM agents to perform different tasks. GraphAide delineates specific *phases* corresponding to the set of features required to complete a user task. As illustrated in Figure 1, the *generation* phase combines information from multiple sources to construct a Knowledge Graph (KG), which serves as a knowledge repository alongside the vector database. In the *query* phase, GraphAide provides a user interface that allows querying the knowledge sources and generating responses in a user-specified format, accompanied by explanations and reasoning.

GraphAide decomposes the query and reasoning research problem into following sub-problems and addresses them using a graph-based RAG approach:

- Data ETL and Normalization
- Named-Entity Recognition
- Domain-aware semantic annotation
- Graph Modeling and Storage
- Query interpretation and context generation

메모

Authorized licensed use limited to: Kookmin University. Downloaded on August 31,2026 at 11:15:48 UTC from IEEE Xplore. Restrictions apply.

- Query expansion and candidate generation
- Summarization and explainable response generation

A. Data Ingestion

Domain-specific datasets are available in multiple structured and unstructured formats, depending on the use cases. GraphAide creates an abstraction layer that allows end users to work with commonly used data formats such as plain text, CSV, PDF, etc. A collection of source data (i.e., files) is ingested into GraphAide, followed by a process of text splitting and chunking to support efficient search over vector data and to manage data larger than the maximum context window size (in number of tokens) of an LLM. GraphAide employs a byte pair encoding (BPE)-based tokenizer [13] that segments input text into sub-word units, enabling the system to handle a wide variety of languages. The BPE-based encoding is also supported by OpenAI embedding models [14] and has been shown to perform well across different query scenarios due to its lossless, reversible, and efficient encoding. Additionally, GraphAide uses the $RecursiveCharacterTextSplitter$, which recursively examines characters in the input data to dynamically identify the optimal splitting strategy. GraphAide employs static $chunk_size$ and $chunk_overlap$ values, as suggested in related work [15], and also allows users to configure the text splitter for specific application domains.

B. Data Storage

Persistence of knowledge curated from multiple sources is critical for the reproducibility and incremental updates of the underlying digital assistant. In contrast to basic RAG, GraphAide persists contextual knowledge in both vector and graph databases. After splitting and chunking, as discussed in Section III-A, the source data is converted into a collection of $documents$, where each $document$ serves as an atomic knowledge source used to answer user queries. In a RAG-based LLM application, *embedding*-based semantic similarity is a key feature. Embeddings are numerical representations of information, where semantically similar content is placed closer together in the embedding space. GraphAide uses a vector database to store embeddings of domain-specific reference datasets or general-purpose knowledge bases, such as WikiData [16]. GraphAide provides a simple API to ingest datasets into the vector database and persist them locally for future use. The novelty of GraphAide lies in its use of a graph database to store additional structural information. GraphAide's API offers domain-specific graph modeling and graph persistence functionality. The graph database is used in subsequent phases to enhance the performance of retrieval operations.

C. Knowledge Graph Generation

Generation of an ontology-guided Knowledge Graph (KG) from heterogeneous sources is one of the key contributions of GraphAide. GraphAide defines a KG generation phase that supports operations such as named entity extraction, relationship extraction, entity disambiguation, KG encoding, and KG maintenance. It employs a prompt-based, few-shot approach to instruct an LLM to generate a knowledge graph. GraphAide supports the dynamic selection of an ontology or a list of node and edge types as input to the generation phase, which restricts the LLM's response schema, ensuring it generates an attributed subgraph for a given input text. For the KG generation phase, an example query could be a paragraph from an unstructured source, such as a news article. GraphAide uses Pydantic [17], an annotation-based schema validator, to maintain the consistency of the generated subgraph throughout the entire KG generation phase.

GraphAide Component	Metrics
Named Entity Mention Extraction	Answer Accuracy
Semantic Annotation	Context Relevancy, Context Accuracy, Answer Relevancy, Answer Accuracy, Semantic Similarity
Entity Disambiguation	Context Precision, Context Recall, ROC Curve
Query Expansion	Answer Relevancy
Graph Query Generation	Answer Accuracy, Answer Relevancy

TABLE I: Metrics for Different Components in GraphAide

D. Subgraph Query Generation

GraphAide's *query phase* begins with a natural language user query, as shown in Figure 1. User queries are often incomplete and imprecise, which can lead to reduced retrieval accuracy and relevance when using vector databases. To address query-document mismatch in sparse retrieval, LLM-based query expansion has been shown to alleviate these challenges [18]. GraphAide applies a similar approach to embedding-based dense retrieval, using a template-based method to generate additional versions of the input query while preserving the key entities and relationships mentioned in the original query. Subsequently, GraphAide leverages LLMs to convert the expanded set of queries into attributed subgraph matching templates, as illustrated in Figure 1. By combining expressive subgraph matching with vector-based semantic similarity, GraphAide enhances retrieval accuracy. The GraphAide API supports commonly used graph databases, such as Neo4J [19], and can auto-generate Cypher queries for the expanded query set. Moreover, the GraphAide API provides different prompt templates to generate queries for domain-specific graph data models based on custom schema.

E. Metrics and Evaluation

Evaluation of a comprehensive GenAI application with multiple components (aka *agents*) requires a collection of relevant quantitative metrics, as a single metric is often insufficient to explain the performance of all components. There is also a need to identify relevant metrics for each agent based on its functionality, role, and the type of data it processes. Both phases of GraphAide involve multiple sub-problems, each with different metric requirements as shown in table I. GraphAide leverages RAGAS [20] to prepare evaluation dataset and compute a collection of relevant quantitative metrics. Future work will focus on developing an API to auto-generate evaluation datasets and metrics. Additionally,

메모

Authorized licensed use limited to: Kookmin University. Downloaded on August 31,2026 at 11:15:48 UTC from IEEE Xplore. Restrictions apply.

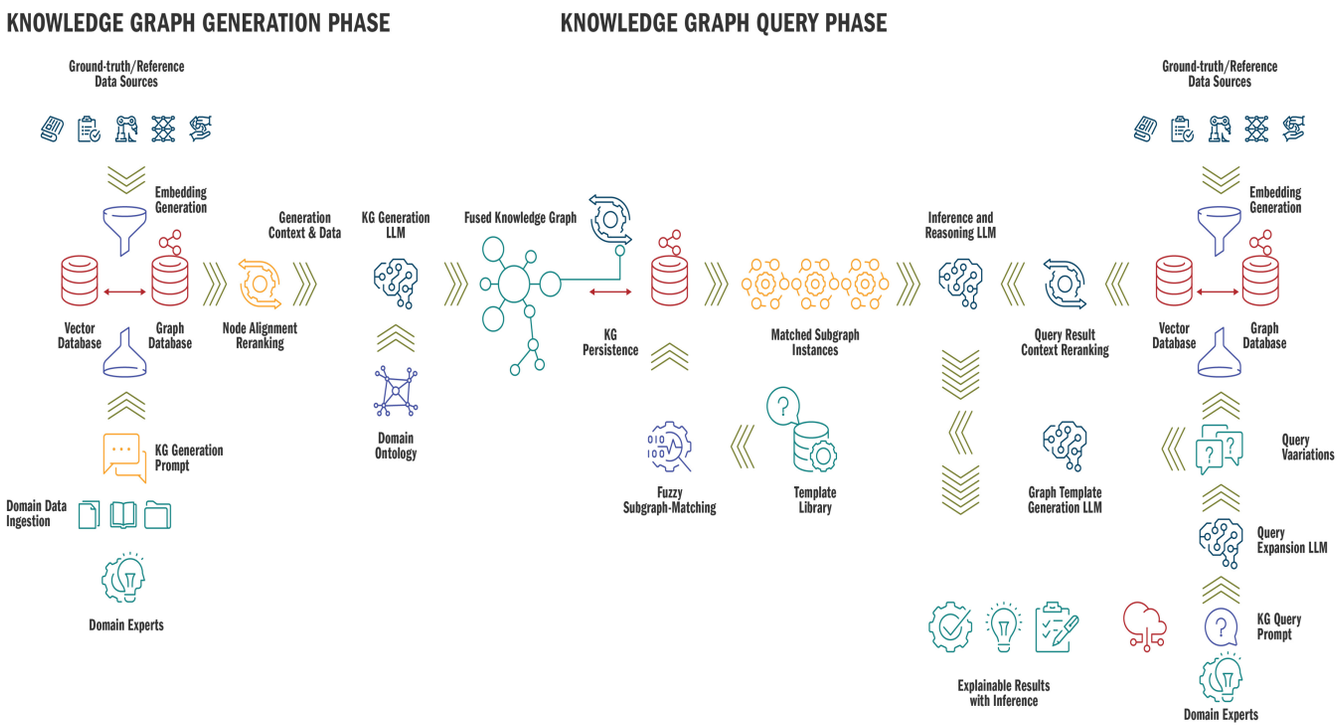


Fig. 1: GraphAide Architecture, representing different phases and agentic workflow

qualitative metrics are required to evaluate generated responses in application domains with a *human-in-the-loop* approach. GraphAide provides methods to configure different retrieval and LLM models, producing comparable results for subject matter expert (SME) evaluation. Examples of different metrics are presented in section IV.

F. Explainable Result Generation

GraphAide simplifies the execution of the templates generated in section III-D for a given graph database and facilitates subgraph matching. It offers an API to combine subgraph matching results with vector-based semantic similarity, creating a local context for the LLM during the query phase. Unlike the KG-generation phase, the query phase uses templates to summarize and explain results from multiple retrieval sources, rather than focusing on entity disambiguation. GraphAide executes multiple parallel requests to the LLM for the expanded query set and makes a secondary call to aggregate all the responses, generating a single coherent response for the user query. Auto-generation of complex templates with multiple constraints and clauses is a part of our future work. We plan to use a few-shot prompt approach to enhance the template generation process.

IV. Experiment

For our experiment, we instantiated GraphAide using OpenAI LLM, Chroma vector database, Neo4j graph database, and LangChain application development framework. We used GPT-4o model that has larger context window and improved reasoning and deduction capabilities. GPT-4o has also shown improved multi-lingual support and that makes it relevant for our use case discussed below. We leveraged the work of two existing DARPA efforts. Active Interpretation of Disparate Alternatives (AIDA) [21] developed multi-hypothesis semantic engines to generates and understand different interpretations of events, situations, and trends from a variety of unstructured sources. Generating Alternative Interpretations for Analysis (GAIA) [22] developed automated solutions to extract entities, emerging events, situations, and trends of interest for Ukraine-Russia political conflict scenario. An ontology [23] is also constructed containing approximately 190 entity types, 140 event types, and 50 relation types. A DARPA version of WikiData (DWD) is also developed as a reference knowledge base for the entity disambiguation process focused on geopolitical scenarios.

To demonstrate GraphAide's capabilities, we used AIDA source data in text format and a Web Ontology Language (OWL) ontology in Terse RDF Triple Language (TURTLE)

Authorized licensed use limited to: Kookmin University. Downloaded on August 31,2026 at 11:15:48 UTC from IEEE Xplore. Restrictions apply.

메모

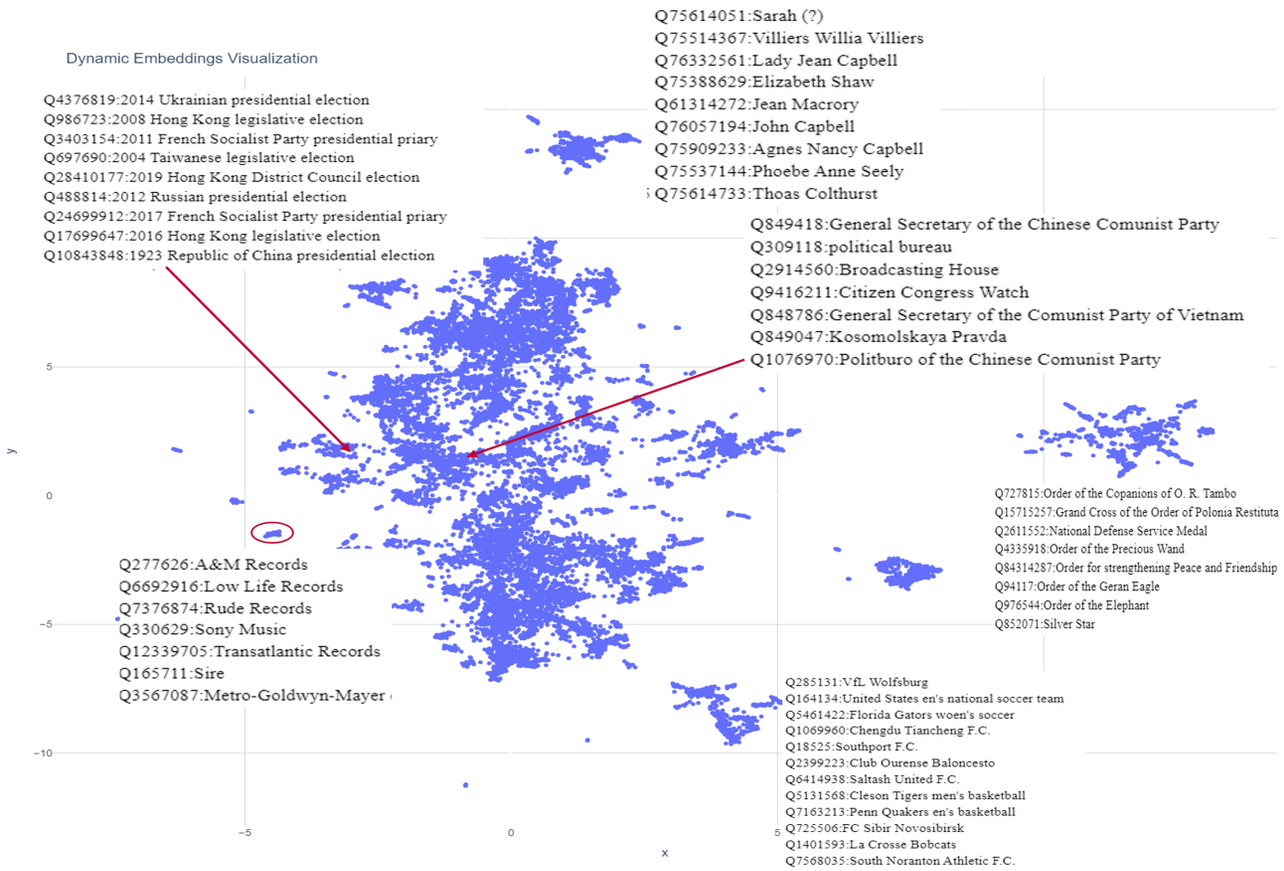


Fig. 2: DARPA WikiData (DWD) Embeddings Used in Retrieval

format to generate a KG. We evaluated the quality of the generated KG by comparing it with a GAIA-generated KG. We also used DWD as a reference knowledge base, stored in Chroma vector database for retrieval during KG generation phase. We curated natural language description of DWD identifiers (i.e., QIDs) and constructed *documents* to store in the vector database. We use QIDs as the document identifier and use LLMs to disambiguate mentions to top-ranked QID. GraphAide also generate UMAP embeddings of vector database to visualize the clusters of topics discussed in the KG, as shown in Figure 2. The embeddings display distinct clusters of concepts such as music, sports, politics, etc. Additionally, they show high similarity between named entities, which is used in the entity disambiguation process. Future work will also incorporate the temporal properties of these named entities to perform disambiguation during the KG generation phase.

GraphAide's *generation* phase processed 1,846 news articles and generated a KG, as shown in Figure 3. An ontology-guided and DWD-based disambiguation agent extracts entity mentions from the text and performs RAG-based node re-ranking to identify the closest matched QID. GraphAide iteratively performed text chunking, generated partial KGs from the given chunks, and loaded them into the Neo4j graph database.

In contrast to the basic RAG, GraphAide's KG-based hybrid-RAG approach provides higher specificity and cross-document inference to construct a richer context for LLM queries. Additionally, GraphAide's modular design allows dynamic selection from multiple retrieval sources for benchmarking purposes. We performed a qualitative evaluation of the generated responses in each phase. For the generation phase, LLM query templates were designed to extract natural language mentions from the input text, perform entity disambiguation using DWD, and perform type assignment (nodes and edges) based on the input ontology. As shown in Figure 3, the KG schema is constrained by the domain-specific ontology. The resulting KG has demonstrated better NER and relation extraction quality than the baseline KG [22], [24]. An extracted node type follows a hierarchical schema to represent the most specific concept about the node. For example, $GPE_UrbanArea_City$ represents a class $City$, which is a subclass of an urban area, itself a subclass of a geopolitical entity. Similarly, $ConflictAttack_FirearmAttack$ represents an attack event related to some conflict that involves firearms. As shown in Figure 5, the baseline KG shows a node type imbalance, where the most frequently extracted type is "PER" and the rest of the node types constitute a small fraction of semantic annotations. Similarly, non-binary relationship

메모

Authorized licensed use limited to: Kookmin University. Downloaded on August 31,2026 at 11:15:48 UTC from IEEE Xplore. Restrictions apply.

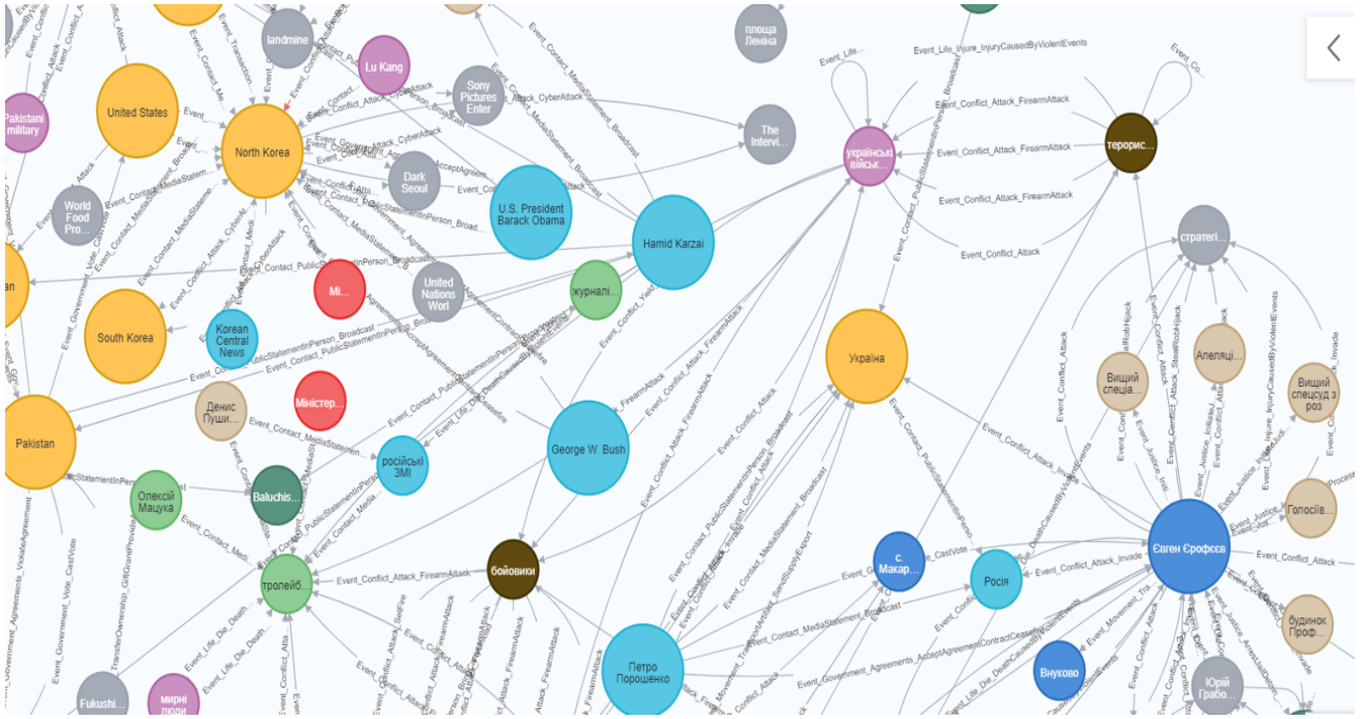


Fig. 3: GraphAide generated KG from News Sources

types such as “Evaluate” and “GeneralAffiliation” dominate the relationship extraction. In contrast, GraphAide extracts a more diverse and dense node type distribution, as shown in Figure 4. Similarly, for the edge types, GraphAide performs better in extracting event-based types, given the input data describes many temporal events.

We evaluated the semantic annotation process of assigning node types using answer relevancy, answer correctness, and semantic similarity metrics, leveraging the RAGAS library. As shown in figure 6, we observed some anomalous node type assignments that cannot be explained using answer relevancy or correctness alone. In such cases, semantic similarity can be used to explain the assignment made by the LLM, as discussed next.

For a user input “The Illinois Traction Building serves dual functions as both a railway terminal and commercial structure in Champaign, USA. The architectural design allows for its dual functionality. As such, the building is a marked example of multi-use infrastructure in the region.”, answer correctness is measured as 0.230700, but further evaluation reveals that the LLM’s prediction about this node, *FAC.Building, ORG.Government.Railway*, is semantically similar to the ground truth, *FAC.Building.OfficeBuilding, FAC.Installation.TrainStation*, which leads to a high semantic similarity of 0.922801.

In the query phase, we used GraphAide to analyze the input data from multiple sources and provide responses to user queries that rely upon the curated knowledge, providing contextual information to the LLM. To demonstrate GraphAide’s modularity, we set up a pipeline to use different levels of augmentation in responding to user queries. In addition to using OpenAI’s GPT-4o as the baseline LLM, we also utilized WikiData, a domain-specific dataset, and an enriched KG from the domain-specific dataset to answer user queries. For the qualitative evaluation of the generated responses, we compared outputs from different augmentation levels. As shown in Figure 7, the specificity of the generated response increases as we enrich the *context* shared with LLMs. Figure 7 shows that a direct query to the LLM endpoint may fail to provide a relevant response either because the LLM is not trained on data about the specific fact or because the fact is more recent than the LLM’s cutoff date. Using a reference KB may improve the response as it grounds the user query into a set of disambiguated entities. Additionally, the use of domain-specific ground-truth documents enables sharing the provenance of the generated response. As shown in Figure 7, the AIDA document corpus contains ground-truth related to the query and allows the LLM to provide a highly relevant response. Taking this a step further, the KG-based retrieval improves *context relevancy* by including entities and concepts spread across documents in the corpus. Entities such as “Donetsk People’s Republic” and “Russian Volunteer” are not necessarily mentioned in the same document in the vector database but are rather connected through a subgraph. The context generated from the subgraph is crucial for the LLM to generate additional evidence and explanations in the response. In addition to the qualitative evaluation based on user feedback, we have also identified individual components of GraphAide suitable for quantitative evaluation to measure the impact of the domain ontology and reference knowledge base. Future work will leverage

Authorized licensed use limited to: Kookmin University. Downloaded on August 31,2026 at 11:15:48 UTC from IEEE Xplore. Restrictions apply.

메모

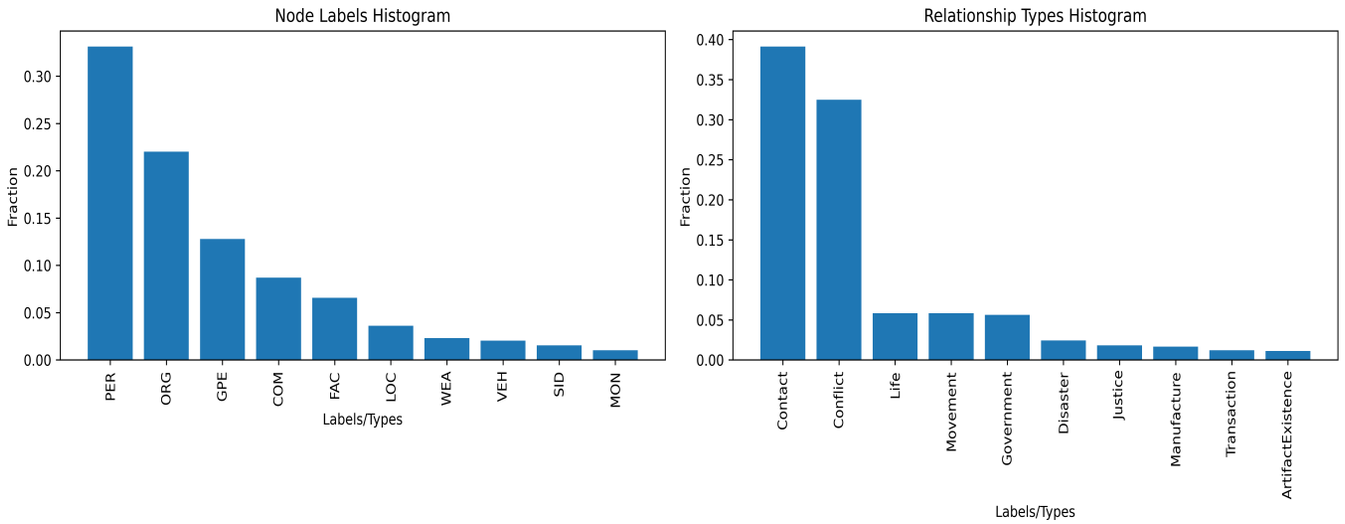


Fig. 4: Type Distributions (Top-10) of GraphAide generated KG

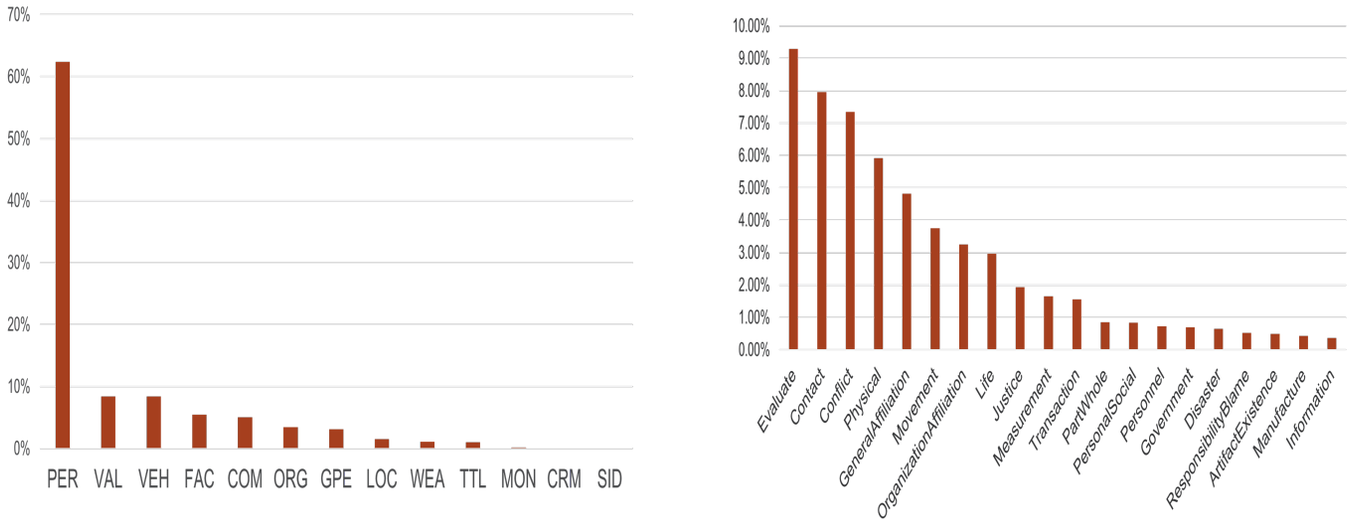


Fig. 5: Type Distributions of Baseline KG. Node (left), Relationship (right)

quantitative metrics such as precision, recall, and retrieval relevancy to measure improvements in entity disambiguation, semantic type assignment, and KG generation tasks for a KG-based hybrid RAG.

V. Conclusion

Large Language Models (LLMs) offer exciting opportunities to develop digital assistents for domain-specific tasks. Advanced Retrieval Augmentation Generation (RAG) addresses explainability, scalability, and usability concerns in using LLMs and increases user confidence in the technology adoption. We present GraphAide, that leverages knowledge graph and subgraph matching capabilities to improve the accuracy of RAG-based applications. We present the reference architecture and its key components. We also demonstrate an instance of GraphAide to generate a Knowledge Graph (KG) from news articles describing historical ukraine-russian political conflict scenario. Future work will focus on formal evaluation to report quantitative improvements in metrics such as accuracy and relevancy.

Acknowledgement

The research described in this paper is partially supported by the Resilience Through Data Driven, Intelligently Designed Control (RD2C) Initiative at Pacific Northwest National Laboratory (PNNL) and the United States federal government. Pacific Northwest National Laboratory is a multiprogram national laboratory operated for the US Department of Energy (DOE) by Battelle Memorial Institute under Contract No. DE-AC05-76RL01830. PNNL Information Release PNNL-SA-205147.

Authorized licensed use limited to: Kookmin University. Downloaded on August 31,2026 at 11:15:48 UTC from IEEE Xplore. Restrictions apply.

메모

메모

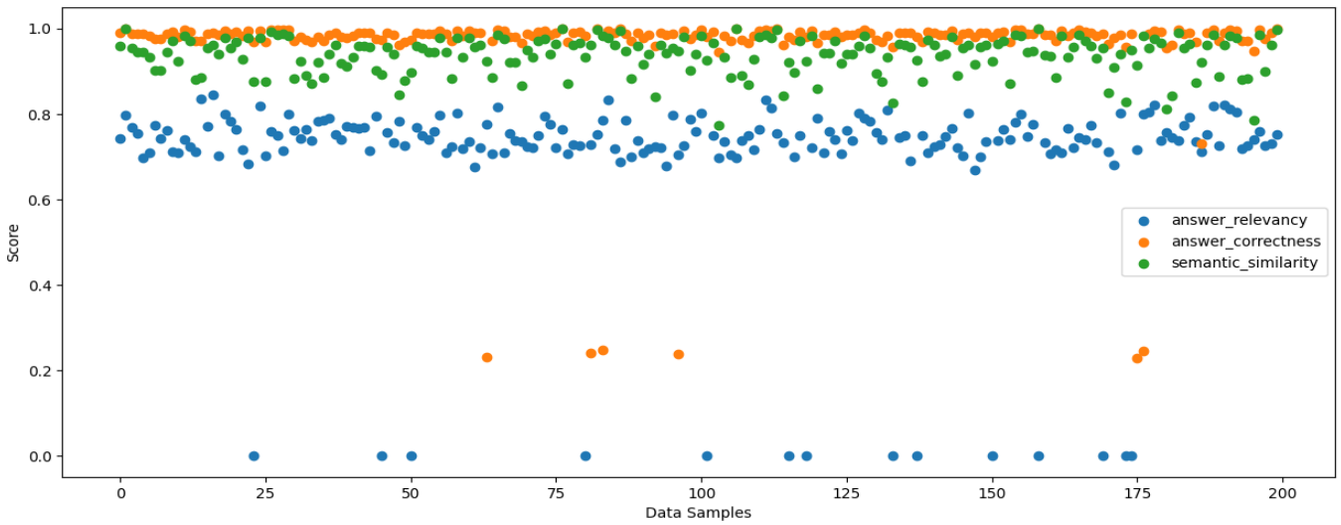


Fig. 6: Evaluation of Semantic Annotation Process

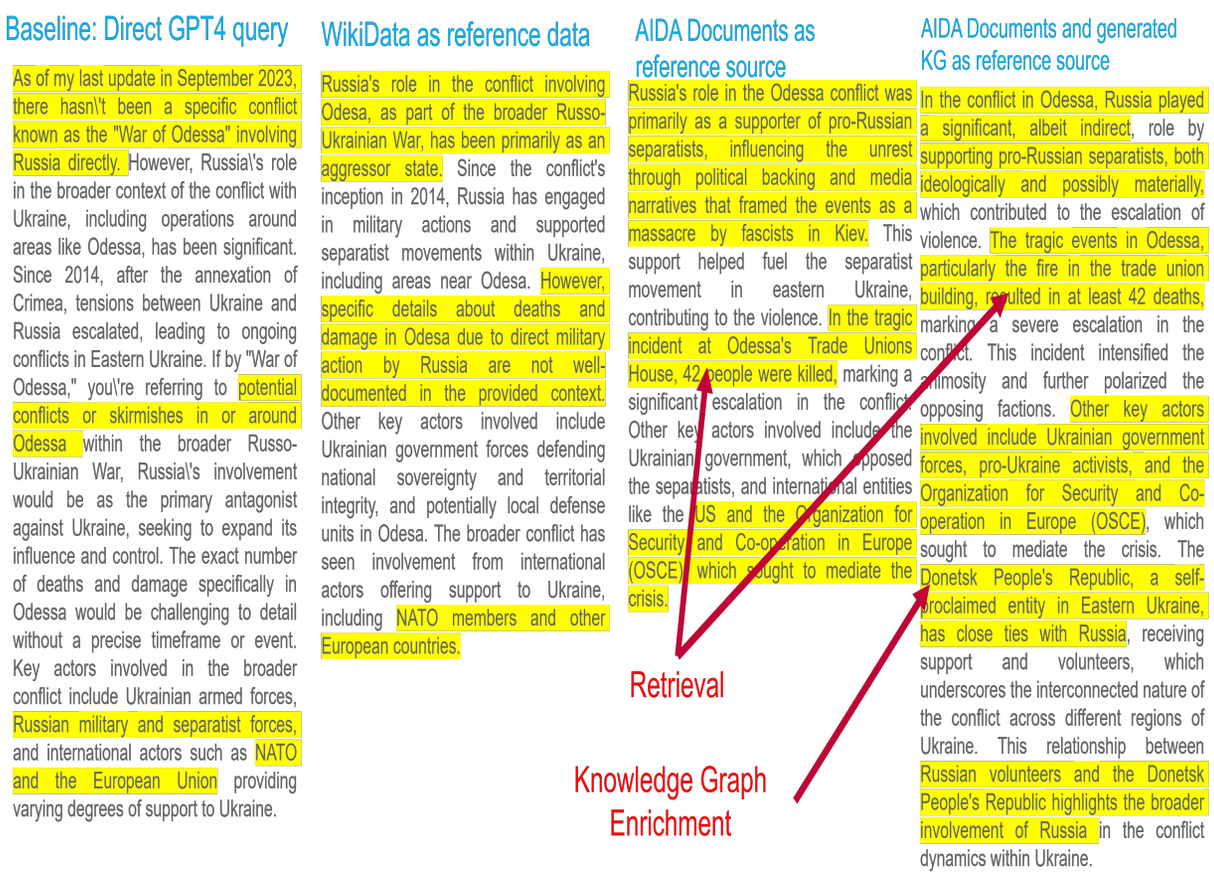


Fig. 7: GraphAide results for a user query ‘Describe the role of Russia in the of war of Odessa in 100 words and list how many deaths and damage happened there. Also list other key actors involved.’

REFERENCES

[1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[2] T. Zhang, F. Ladhak, E. Durmus, P. Liang, K. McKeown, and T. B. Hashimoto, “Benchmarking large language models for news summarization,” *Transactions of the Association for Computational Linguistics*,

Authorized licensed use limited to: Kookmin University. Downloaded on August 31,2026 at 11:15:48 UTC from IEEE Xplore. Restrictions apply.

vol. 12, pp. 39–57, 2024.
[3] T. Berners-Lee, J. Hendler, and O. Lassila, “The semantic web: A new form of web content that is meaningful to computers will unleash a revolution of new possibilities,” in *Linking the World’s Information: Essays on Tim Berners-Lee’s Invention of the World Wide Web*, 2023, pp. 91–103.
[4] C. Bizer, T. Heath, and T. Berners-Lee, “Linked data-the story so far,” in *Linking the World’s Information: Essays on Tim Berners-Lee’s Invention of the World Wide Web*, 2023, pp. 115–143.
[5] A. Bosselut, H. Rashkin, M. Sap, C. Malaviya, A. Celikyilmaz, and Y. Choi, “Comet: Commonsense transformers for automatic knowledge graph construction,” *arXiv preprint arXiv:1906.05317*, 2019.
[6] J.-Y. Yao, K.-P. Ning, Z.-H. Liu, M.-N. Ning, and L. Yuan, “Llm lies: Hallucinations are not bugs, but features as adversarial examples,” *arXiv preprint arXiv:2310.01469*, 2023.
[7] N. McKenna, T. Li, L. Cheng, M. J. Hosseini, M. Johnson, and M. Steedman, “Sources of hallucination by large language models on inference tasks,” *arXiv preprint arXiv:2305.14552*, 2023.
[8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
[9] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023.
[10] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A graph rag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
[11] M. Trajanoska, R. Stojanov, and D. Trajanov, “Enhancing knowledge graph construction using large language models,” *arXiv preprint arXiv:2305.04676*, 2023.
[12] T. Ban, L. Chen, X. Wang, and H. Chen, “From query tools to causal architects: Harnessing large language models for advanced causal discovery from data,” *arXiv preprint arXiv:2306.16902*, 2023.
[13] P. Gage, “A new algorithm for data compression,” *The C Users Journal*, vol. 12, no. 2, pp. 23–38, 1994.
[14] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, “Language models are unsupervised multitask learners,” *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
[15] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” *arXiv preprint arXiv:2004.04906*, 2020.
[16] D. Vrandečić and M. Krötzsch, “Wikidata: a free collaborative knowledgebase,” *Communications of the ACM*, vol. 57, no. 10, pp. 78–85, 2014.
[17] S. Colvin, “Pydantic,” 2023, version 2.x. [Online]. Available: https://github.com/pydantic/pydantic
[18] M. A. K. Ayoub, Z. Su, and Q. Li, “A case study of enhancing sparse retrieval using llms,” in *Companion Proceedings of the ACM on Web Conference 2024*, 2024, pp. 1609–1615.
[19] Neo4j, Inc., *Neo4j Graph Database*, Neo4j, Inc., San Mateo, CA, 2024, available at: https://neo4j.com. [Online]. Available: https://neo4j.com
[20] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2023.
[21] “Active interpretation of disparate alternatives (aida),” https://www.darpa.mil/program/active-interpretation-of-disparate-alternatives, accessed: 2024-10-09.
[22] T. Zhang, A. Subburathinam, G. Shi, L. Huang, D. Lu, X. Pan, M. Li, B. Zhang, Q. Wang, S. Whitehead *et al.*, “Gaia-a multi-media multilingual knowledge extraction and hypothesis generation system.” in *TAC*, 2018.
[23] Next Century Corporation, “Aida interchange format,” https://github.com/NextCenturyCorporation/AIDA-Interchange-Format, 2023, accessed: 2024-10-09.
[24] S. Purohit, N. Van, and G. Chin, “Semantic property graph for scalable knowledge graph analytics,” in *2021 IEEE International Conference on Big Data (Big Data)*, 2021, pp. 2672–2677.

메모

Authorized licensed use limited to: Kookmin University. Downloaded on August 31,2026 at 11:15:48 UTC from IEEE Xplore. Restrictions apply.